

Memoria del Proyecto Desoft

Joel Amorín Rodríguez

Odei Alcalde Rodríguez
Lucas Arestíño Lorenzo

Pablo Araújo Rodríguez

17 de mayo de 2026

Índice

Memoria del Proyecto: Diseño de Software	1
Introducción	1
1. Fase de Inicio	1
2. Fase de Elaboración	2
2.1 Modelo de Casos de Uso	2
CASOS DE USO DE PRIORIDAD ALTA	3
CASOS DE USO DE PRIORIDAD MEDIA	10
2.2 Modelo de Vocabulario	13
3. Fase de Construcción	15
3.1 Planificación	15
3.2 Iteración 1	15
3.3 Iteración 2	17
4. Fase de Transición	18

Memoria del Proyecto: Diseño de Software

Equipo de Desarrollo (GR1_J2): * Joel Amorín Rodríguez * Odei Alcalde Rodríguez * Lucas Arestíño Lorenzo * Pablo Araújo Rodríguez

Introducción

Este documento constituye la memoria del Proyecto de Diseño correspondiente a la asignatura de Diseño de Software (GrEl) del curso 2025/2026. En las siguientes páginas se relata la toma de decisiones y la evolución de la arquitectura del proyecto a lo largo de todo el cuatrimestre. Para ello, la documentación se ha estructurado siguiendo lo mejor posible las directrices y lecciones dadas a lo largo del curso. A su vez, hemos dividido nuestra experiencia en cuatro etapas fundamentales: Fase de Inicio, Fase de Elaboración, Fase de Construcción y Fase de Transición.

1. Fase de Inicio

Dado el contexto de este proyecto, la Fase de Inicio tuvo una duración muy breve, ocupando apenas la primera sesión de prácticas. Durante este tiempo, el trabajo del equipo se centró en analizar con

detenimiento el enunciado del proyecto y en aclarar directamente con el profesor aquellos aspectos sobre la funcionalidad deseada que generaban algún tipo de duda inicial.

Para acotar adecuadamente el marco de trabajo, a continuación se detalla la justificación y los límites del sistema desarrollado:

El propósito fundamental de este proyecto es poner en práctica y consolidar los conocimientos adquiridos en la asignatura mediante el diseño y desarrollo completo de un juego de estrategia por turnos de estilo Age of Empires. El sistema sirve como vehículo para aplicar los principios del diseño dados en clase de teoría, elaborando en la medida de lo posible un trabajo escalable y basado en el uso de patrones de diseño.

El alcance del software desarrollado comprende la simulación completa de un entorno de juego capaz de soportar hasta 4 participantes o civilizaciones. El sistema se encarga de la generación, visualización y actualización de un mapa interactivo representado mediante una interfaz gráfica ASCII, la cual simula niebla de guerra mostrando únicamente la porción de terreno .

A nivel mecánico, el alcance abarca la recolección y gestión de recursos (madera, piedra y comida), la construcción de infraestructuras (ciudadelas, cuarteles, casas y torres), la generación, movimiento y agrupación de personajes (tanto paisanos como soldados). Asimismo, el sistema incluye la gestión de combates entre facciones rivales, la automatización de defensas (ataques de torres) y la persistencia de datos, permitiendo al administrador guardar y cargar el estado íntegro de la partida en cualquier momento.

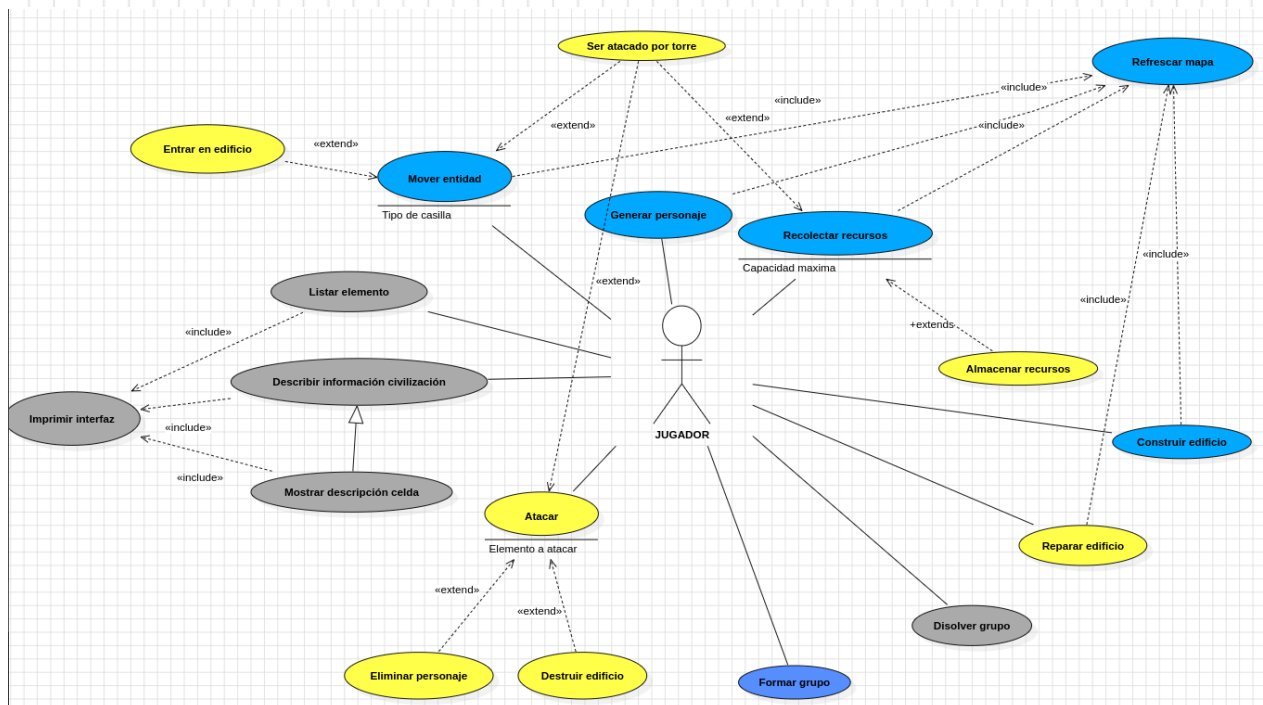
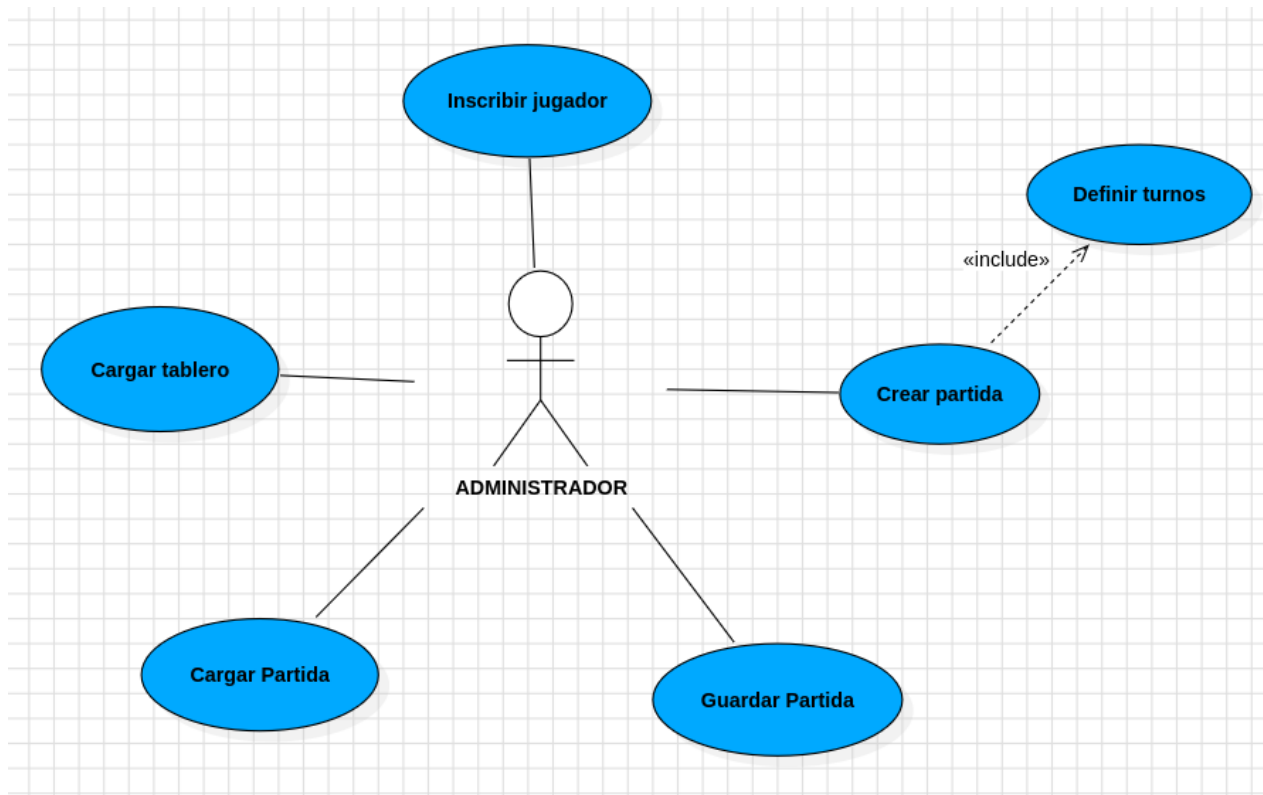
2. Fase de Elaboración

La Fase de Elaboración tuvo lugar durante cuatro semanas que incluyeron tres sesiones de prácticas. En ellas se llevaron a cabo tareas de análisis. En primer lugar, se desarrolló un modelo de comportamiento que hace posible expresar lo que tiene que hacer el sistema sin entrar en la forma de conseguirlo. Justo antes de la primera entrega de seguimiento, se elaboró un modelo de vocabulario que debía atrapar fielmente la estructura del dominio de aplicación.

2.1 Modelo de Casos de Uso

En primera instancia, el modelo de casos de uso permite especificar los requisitos funcionales del sistema y establecer la frontera entre dicho sistema y su entorno, motivo por el que resulta de gran utilidad como contrato entre el cliente y el equipo de desarrollo.

A continuación se incluyen los diagramas generales y las descripciones completas de todos los casos de uso identificados (Azul: Prioridad Alta, Amarillo: Prioridad Media, Gris: Prioridad Baja):



CASOS DE USO DE PRIORIDAD ALTA

CASO DE USO: CU00 - Crear partida

- **Actores:** Administrador
- **Resumen:** Inicia formalmente el juego una vez que el entorno y los participantes están listos.

Se encarga de ensamblar el estado inicial, definir el sistema de turnos y ceder el control al primer jugador para que comience la acción.

- **Precondiciones:** -
- **Postcondiciones:** La partida pasa a estado “Iniciada”. La interfaz gráfica ASCII se renderiza por primera vez desde la perspectiva del jugador con el turno.
- **Escenario Principal:**
 - a. El administrador solicita iniciar la partida.
 - b. Se instancia la partida.
 - c. Se carga el tablero con el método deseado por el administrador.
 - d. Se inscriben los jugadores de la partida.
 - e. El sistema define la política de turnos de la nueva partida.
 - f. El sistema establece el turno activo en el primer jugador de la lista.
 - g. La partida arranca y la interfaz muestra el mapa visible para el jugador activo.
- **Escenario Alternativo 1:** Requisitos insuficientes para iniciar. En el paso “b”, si no hay tablero cargado o no hay suficientes jugadores inscritos, el sistema advierte al administrador de lo que falta por configurar y aborta el inicio de la partida.
- **Extensiones:** -
- **Inclusiones:** Incluye al caso de uso: “Definir turnos”.
- **Prioridad:** Alta

CASO DE USO: CU01 - Refrescar mapa

- **Actores:** Administrador
- **Resumen:** Se actualiza la representación gráfica del terreno en la interfaz ASCII, reflejando el estado actual de los elementos y recursos, mostrándose únicamente desde la perspectiva del jugador que tiene el turno activo.
- **Precondiciones:** La partida fue iniciada y los turnos definidos. Se debe haber producido una acción que altere la posición o el estado de algún elemento (movimiento de personajes, creación o eliminación de edificios/personajes...)
- **Postcondiciones:** La interfaz ASCII muestra el mapa reestructurado y actualizado, siendo visibles únicamente las celdas que haya visitado el jugador actual (así como sus adyacentes).
- **Escenario Principal:**
 - a. El sistema identifica la necesidad de refrescar el mapa al producirse una acción que lo altere.
 - b. El sistema interpreta el origen de la actualización del mapa.
 - c. El sistema actualiza los caracteres ASCII de las celdas visibles según los cambios pertinentes.
 - d. El sistema muestra por pantalla el mapa actualizado (solamente las celdas visibles).
- **Prioridad:** Alta

CASO DE USO: CU02 - Mover entidad

- **Actores:** Jugador
- **Resumen:** Permite desplazar un personaje (o grupo de personajes) perteneciente a la civilización activa a una celda adyacente válida del mapa, respetando las restricciones de transitabilidad, tipo de unidad y alcance de movimiento. El movimiento puede desencadenar efectos automáticos como ataques de torres enemigas o activación de visibilidad en nuevas celdas.

- **Precondiciones:** La partida está iniciada. Es el turno del jugador activo. El personaje o grupo seleccionado pertenece a la civilización activa. La celda destino está dentro de los límites del mapa. La celda destino es transitable (pradera). Si se trata de un caballero, puede desplazarse hasta dos celdas; el resto solo una.
- **Postcondiciones:** El personaje/grupo cambia su posición a la celda destino. Se actualiza la visibilidad del mapa para el jugador activo (se revelan nuevas celdas visitadas y adyacentes). Si existen torres enemigas adyacentes, pueden ejecutarse ataques automáticos. Si el personaje entra en el rango de ataque de enemigos, puede recibir daño.
- **Escenario Principal:**
 - a. El jugador selecciona el personaje o grupo que desea mover.
 - b. El sistema muestra las direcciones posibles (NORTE, SUR, ESTE, OESTE).
 - c. El jugador indica la dirección de desplazamiento.
 - d. El sistema comprueba que la celda destino es válida y transitable.
 - e. El sistema actualiza la posición del personaje/grupo.
 - f. El sistema recalcula la visibilidad del mapa.
- **Escenario Alternativo 1:** En el paso “d”, si la celda no es transitable o está ocupada por un edificio no accesible o recurso, el sistema muestra un mensaje de error y vuelve al paso “b”.
- **Escenario Alternativo 2:** Si el personaje intenta desplazarse más celdas de las permitidas (por ejemplo, una unidad distinta al caballero intenta mover dos), el sistema cancela la acción y muestra un mensaje de error.
- **Escenario Alternativo 3:** Si tras el movimiento el personaje entra en una celda adyacente a una torre enemiga, la torre ejecuta un ataque automático conforme a las reglas del juego.
- **Inclusiones:** Incluye al caso de uso: “Refrescar mapa”.
- **Prioridad:** Alta

CASO DE USO: CU03 - Construir Edificio

- **Actores:** Jugador
- **Resumen:** La acción habilita al jugador a construir un edificio del tipo seleccionado (casa, cuartel, ciudadela, torre) en una parcela dada si acaso este dispone de los materiales necesarios (madera y piedra) y dicha casilla es edificable. La construcción del edificio será llevada a cabo por los paisanos disponibles en ese momento. Una vez el edificio sea levantado la celda que ocupa dejará de ser transitable.
- **Precondiciones:** El personaje constructor es un paisano, la celda es edificable y se dispone de la cantidad de materiales necesarios. Además el jugador que desee edificar deberá edificar durante su turno de juego.
- **Postcondiciones:** La celda pasa a contener un edificio (del tipo especificado por el usuario previo a su construcción) y ser no transitable, y los recursos usados para construir son restados del saldo de tesorería del jugador. El paisano quedará liberado de sus tarea, quedando disponible para cualquier otra.
- **Escenario Principal:**
 - a. Se elige un paisano, una celda y el tipo de edificio que se pretende construir (casa, ciudadela, cuartel o torre).
 - b. El sistema comprueba si la celda es edificable.
 - c. El sistema comprueba si hay recursos suficientes (madera y piedra).
 - d. Si acaso se cumple con estas condiciones se procede a restar del saldo del jugador los recursos necesarios para la construcción del edificio.
 - e. Se inicia la construcción del edificio elegido en la celda seleccionada.

- f. Tras un periodo de tiempo determinado se finaliza la tarea y la parcela pasa a estar ocupada por el nuevo edificio.
- g. El paisano queda liberado para llevar a cabo otras labores.
- **Escenario Alternativo 1:** Recursos insuficientes. En el paso “c”, si no se dispone de la cantidad de recursos necesarios, el sistema rechaza la operación y no se construye el edificio.
- **Escenario Alternativo 2:** La casilla donde se desea construir el edificio no es edificable y por lo tanto se impide la acción.
- **Inclusiones:** Incluye al caso de uso: “Refrescar mapa”.
- **Prioridad:** Alta

CASO DE USO: CU04 - Generar Personaje

- **Actores:** Jugador
- **Resumen:** Permite a la civilización activa generar un nuevo personaje (paisano o soldado) desde un edificio habilitado (ciudadela o cuartel), siempre que se disponga de recursos suficientes y exista espacio disponible tanto en el edificio como en las celdas adyacentes.
- **Precondiciones:** La partida está iniciada. Es el turno del jugador activo. El edificio seleccionado pertenece a la civilización activa. El edificio es del tipo adecuado: ciudadela genera paisanos y cuartel genera soldados (legionario, caballero o arquero). Existen recursos suficientes (comida y, si procede, otros costes definidos). Existe al menos una celda adyacente libre para ubicar al personaje. No se supera el límite de alojamiento permitido por las casas.
- **Postcondiciones:** Se descuentan los recursos correspondientes de la civilización. Se crea una nueva instancia del personaje con atributos iniciales por defecto (salud completa, energía inicial, ataque y defensa según tipo). El personaje se posiciona en una celda adyacente libre. Se actualiza el número de integrantes alojados en casas (si aplica).
- **Escenario Principal:**
 - a. El jugador selecciona un edificio propio (ciudadela o cuartel).
 - b. El sistema muestra los tipos de personaje disponibles según el edificio.
 - c. El jugador selecciona el tipo de personaje a generar.
 - d. El sistema comprueba disponibilidad de recursos.
 - e. El sistema verifica que existe una celda adyacente libre.
 - f. El sistema crea el personaje con sus atributos base.
 - g. El personaje se ubica en una celda adyacente válida.
 - h. El sistema descuenta los recursos correspondientes.
- **Escenario Alternativo 1:** En el paso “d”, si no se dispone de comida suficiente (o del coste requerido), el sistema cancela la acción y muestra un mensaje de error.
- **Escenario Alternativo 2:** En el paso “e”, si no hay celdas adyacentes libres o se supera el límite de alojamiento permitido, el sistema cancela la acción.
- **Escenario Alternativo 3:** Si se intenta generar un tipo de unidad no permitido por el edificio seleccionado, el sistema rechaza la acción.
- **Inclusiones:** Incluye al caso de uso: “Refrescar mapa”.
- **Prioridad:** Alta

CASO DE USO: CU05 - Definir turnos

- **Actores:** Administrador
- **Resumen:** Permite al administrador establecer la política que seguirán los turnos de la partida.
- **Precondiciones:** Que se haya creado una partida y se hayan creado al menos 2 participantes.

- **Postcondiciones:** Queda establecido el funcionamiento de los turnos, así como el orden. Una vez comenzado el sistema de turnos las órdenes transmitidas sólo afectarán a la civilización activa y el mapa se dibujará siempre desde la perspectiva del jugador con el turno.
- **Escenario Principal:**
 - a. El sistema comprueba si se han creado al menos dos participantes.
 - b. El administrador establece el orden de participación.
 - c. El administrador establece la política de cambio de turnos. El administrador decide si el cambio es automático o se hace a petición del jugador activo.
 - d. Si se establecen turnos por tiempo se establece la duración de los turnos, si se establecen por número de acciones, se establece el número de acciones; si se establece la cesión del turno por acción del jugador con el turno, no hace falta establecer ningún valor.
- **Escenario Alternativo 1:** No hay suficientes jugadores para poder definir turnos.
- **Escenario Alternativo 2:** Se establece el tiempo de duración de cada turno.
- **Escenario Alternativo 3:** Se establece el número de acciones de cada turno.
- **Escenario Alternativo 4:** Se establece el paso de turno por acción del jugador con el turno.
- **Prioridad:** Alta

CASO DE USO: CU06 - Recolectar recursos

- **Actores:** Jugador
- **Resumen:** Se establece la posibilidad de recolectar recursos de una casilla concreta del tablero por parte de uno de los paisanos. En función del tipo de casilla a explotar el jugador puede tomar diferentes tipos de recursos como madera de los bosques, piedra de las canteras o comida de los arbustos. La capacidad de recolección de los paisanos será la que en última instancia determine la reducción de unidades de recursos ligado a la celda contenedora.
- **Precondiciones:** Que sea tu turno. La casilla es adyacente al paisano que recolecta allí. Existe el suficiente espacio de almacenamiento para dar cabida a los recursos recolectados. Además se debe disponer de un paisano o un grupo de ellos para la recolección.
- **Postcondiciones:** Quedan almacenados los recursos en el inventario del paisano y se restan de la casilla de la que fueron extraídos. Si el contenedor del recurso consume sus unidades, la celda pasará a ser una pradera transitable.
- **Escenario Principal:**
 - a. El jugador selecciona a uno de sus paisanos disponibles.
 - b. Se selecciona la casilla correspondiente de la que se quieren extraer los recursos.
 - c. La cantidad de recursos recolectados se fija proporcionalmente a la capacidad actual de recolección del personaje.
 - d. Lo recolectado se almacena en el inventario del paisano y se resta dicha cantidad de los recursos existentes en la casilla.
 - e. El personaje queda liberado de la tarea de recolección y pasa a estar disponible para otras acciones.
- **Escenario Alternativo 1:** El paisano seleccionado no está disponible, por lo que se rechaza la acción.
- **Escenario Alternativo 2:** El paisano seleccionado se encuentra en el máximo de capacidad de recolección (inventario lleno), por lo que no puede recolectar más recursos y debe almacenar los que posee en una ciudadela previo a recolectar nuevas unidades.
- **Escenario Alternativo 3:** El paisano llena su capacidad de almacenamiento mientras se encuentra recolectando. Frena la recolección, debiendo almacenar los recursos en una ciudadela.

- **Escenario Alternativo 4:** Si se agotan los recursos el paisano deja de recolectar, la celda pasa a ser pradera y se actualiza el mapa.
- **Inclusiones:** Incluye al caso de uso: “Refrescar mapa”.
- **Prioridad:** Alta

CASO DE USO: CU07 - Formar grupo

- **Actores:** Jugador
- **Resumen:** Permite al jugador agrupar varios personajes que se encuentran en la misma celda del mapa para que actúen como una única unidad. El grupo resultante tendrá capacidades de ataque, defensa y recolección equivalentes a la suma de las capacidades de sus integrantes. Mientras los personajes estén agrupados no podrán ser gestionados individualmente.
- **Precondiciones:** La partida está iniciada. Es el turno del jugador activo. Los personajes seleccionados pertenecen a la civilización activa. Todos los personajes seleccionados se encuentran en la misma celda del mapa. Los personajes no forman ya parte de otro grupo.
- **Postcondiciones:** Se crea un grupo compuesto por los personajes seleccionados. Las capacidades del grupo (ataque, defensa y, si procede, recolección) pasan a ser la suma de las capacidades individuales de sus integrantes. Los personajes dejan de ser referenciables individualmente mientras formen parte del grupo. El grupo puede realizar acciones como moverse, atacar, cobijarse en edificios o recolectar recursos si incluye paisanos.
- **Escenario Principal:**
 - a. El jugador selecciona varios personajes de su civilización situados en una misma celda.
 - b. El jugador solicita al sistema la creación de un grupo.
 - c. El sistema verifica que todos los personajes seleccionados pertenecen al jugador activo y están en la misma celda.
 - d. El sistema comprueba que ninguno de los personajes pertenece ya a otro grupo.
 - e. El sistema crea una nueva entidad de grupo que contiene a los personajes seleccionados.
 - f. El sistema calcula las capacidades del grupo sumando las capacidades de ataque, defensa y otras habilidades de sus integrantes.
 - g. El sistema registra el grupo como unidad operativa única para futuras acciones.
- **Escenario Alternativo 1:** Personajes en distintas celdas. En el paso “c”, si los personajes seleccionados no se encuentran en la misma celda, el sistema rechaza la operación y muestra un mensaje de error.
- **Escenario Alternativo 2:** Personaje ya perteneciente a otro grupo. En el paso “d”, si alguno de los personajes ya forma parte de otro grupo, el sistema cancela la creación del grupo y notifica el error.
- **Escenario Alternativo 3:** Selección inválida de personajes. Si alguno de los personajes no pertenece a la civilización activa o no está disponible, el sistema rechaza la acción.
- **Inclusiones:** Incluye “Refrescar mapa” (si la representación del grupo altera la visualización del mapa).
- **Prioridad:** Alta

CASO DE USO: CU08 - Cargar tablero

- **Actores:** Administrador
- **Resumen:** Permite configurar y generar la representación espacial en la que se desarrollará la partida. El sistema establece la cuadrícula, los tipos de celda (bosque, cantera, arbusto, pradera) y los recursos asociados.

- **Precondiciones:** El sistema debe estar iniciado.
- **Postcondiciones:** El mapa queda cargado en la memoria del sistema y listo para ubicar a los jugadores, pero la partida aún no ha comenzado.
- **Escenario Principal:**
 - a. El administrador solicita al sistema la carga o generación de un tablero.
 - b. El sistema solicita seleccionar el método de configuración del territorio.
 - c. El administrador elige una opción: adoptar tablero por defecto, generar un nuevo territorio aleatoriamente o configurar a partir de información previamente almacenada.
 - d. El sistema procesa la opción elegida, genera la cuadrícula y distribuye los recursos.
 - e. El sistema confirma que el tablero se ha cargado correctamente.
- **Escenario Alternativo 1:** Error en el archivo de tablero. Si en el paso “c” el administrador escoge configurar el mapa desde información almacenada y el archivo está corrupto o no existe, el sistema muestra un mensaje de error.
- **Prioridad:** Alta

CASO DE USO: CU09 - Inscribir jugador

- **Actores:** Administrador
- **Resumen:** Registra a un nuevo participante en la partida, asignándole una civilización única y ubicando en el mapa sus elementos iniciales por defecto (una ciudadela, un paisano y los recursos centralizados).
- **Precondiciones:** El tablero debe haber sido generado previamente (mediante “Cargar tablero”). El número actual de jugadores inscritos debe ser menor a 4.
- **Postcondiciones:** El jugador queda registrado en el sistema. Su ciudadela y su paisano aparecen ubicados en celdas válidas del mapa.
- **Escenario Principal:**
 - a. El administrador solicita inscribir un nuevo jugador en la sesión actual.
 - b. El sistema verifica que no se ha alcanzado el límite máximo de 4 jugadores.
 - c. El sistema asigna automáticamente una civilización diferente al nuevo jugador.
 - d. El sistema ubica una ciudadela y un paisano pertenecientes a esta civilización en una zona inicial del mapa.
 - e. El sistema otorga un conjunto de recursos iniciales a la civilización.
 - f. El sistema confirma la inscripción exitosa del participante.
- **Escenario Alternativo 1:** Límite de jugadores alcanzado. En el paso “b”, si ya hay 4 jugadores inscritos, el sistema deniega la acción informando que se ha alcanzado el límite máximo de participantes y cancela el caso de uso.
- **Prioridad:** Alta

CASO DE USO: CU10 - Cargar partida

- **Actores:** Administrador
- **Resumen:** Recupera una partida previamente guardada, restaurando por completo el mapa, el inventario de las civilizaciones, la ubicación de las entidades y cediendo el turno al jugador que lo tenía en el momento del guardado.
- **Precondiciones:** El sistema debe estar iniciado. Debe existir al menos un registro de partida guardada válido.
- **Postcondiciones:** Se descarta cualquier estado anterior en memoria. La partida queda en estado “En curso” exactamente como se dejó al guardarla.
- **Escenario Principal:**

- a. El administrador solicita cargar una partida guardada.
- b. El sistema valida la integridad del archivo seleccionado.
- c. El sistema reconstruye el tablero, las civilizaciones, las entidades y el orden de los turnos en memoria.
- d. El sistema actualiza la interfaz ASCII mostrando el mapa desde la perspectiva del jugador al que le corresponde el turno recuperado.
- **Escenario Alternativo 1:** Archivo corrupto o incompatible. En el paso “d”, si el archivo ha sido modificado externamente, está corrupto o pertenece a una versión antigua incompatible, el sistema muestra un error de lectura y vuelve al menú principal o al paso “b”.
- **Prioridad:** Alta

CASO DE USO: CU17 - Almacenar recursos

- **Actores:** Jugador
 - **Resumen:** Permite a un paisano que ha llenado su inventario tras extraer materiales en el mapa, depositar la carga recolectada en una estructura de almacenamiento (como una ciudadela) para que pase a formar parte de la reserva global de la civilización.
 - **Precondiciones:** Es el turno del jugador activo. Un paisano dispone de recursos recolectados en su capacidad personal y se sitúa en una celda adyacente a una ciudadela.
 - **Postcondiciones:** El inventario individual del paisano queda vacío y la cantidad de recursos que portaba se suma a la tesorería de la civilización. El paisano vuelve a estar habilitado para nuevas recolecciones.
 - **Escenario Principal:**
 - a. El jugador selecciona a un paisano que acarrea recursos.
 - b. Se indica como destino una ciudadela aliada adyacente.
 - c. El jugador ejecuta la acción de almacenar.
 - d. El sistema suma los recursos del paisano al inventario global del jugador.
 - e. La capacidad de carga del paisano se vacía, permitiéndole retomar su labor.
 - **Escenario Alternativo 1:** Acción desencadenada automáticamente. En caso de que el sistema fuerce al jugador a almacenar antes de seguir, esta acción salta como un requisito forzoso cuando se alcanza la capacidad máxima de recolección en campo.
 - **Extensiones:** Extiende al caso de uso: “Recolectar recursos”.
 - **Inclusiones:** Incluye al caso de uso: “Refrescar mapa”.
 - **Prioridad:** Alta
-

CASOS DE USO DE PRIORIDAD MEDIA

CASO DE USO: CU09 - Guardar partida

- **Actores:** Administrador
- **Resumen:** Permite almacenar el estado íntegro actual del juego (distribución del mapa, recursos restantes, posiciones de personajes/edificios, estado de las civilizaciones y el orden de los turnos) para poder reanudar el enfrentamiento en otro momento.
- **Precondiciones:** Debe haber una partida “En curso”.
- **Postcondiciones:** Se genera un archivo o registro persistente con toda la información de la partida. El juego puede continuar con normalidad tras el guardado.
- **Escenario Principal:**
 - a. El administrador interrumpe el juego y solicita guardar la partida.

- b. El sistema recopila el estado de todas las celdas, recursos, entidades y turnos.
- c. El sistema escribe la información en un archivo de guardado.
- d. El sistema notifica que el guardado se ha realizado con éxito.
- **Escenario Alternativo 1:** Error de escritura. En el paso “e”, si el sistema no tiene permisos o no hay espacio de almacenamiento, se muestra un error advirtiéndole que la partida no pudo ser guardada y se devuelve el control al administrador.
- **Prioridad:** Media

CASO DE USO: CU11 - Entrar en edificio

- **Actores:** Jugador
- **Resumen:** Permite a un personaje o grupo de personajes introducirse en un edificio aliado (como una ciudadela o cuartel) para protegerse, desapareciendo temporalmente del mapa visible y ocupando el espacio de alojamiento disponible en el interior de la estructura.
- **Precondiciones:** La partida está iniciada y es el turno del jugador activo. El personaje o grupo pertenece a la civilización activa y se ha desplazado hacia una celda ocupada por un edificio aliado. El edificio tiene capacidad de guarnición o alojamiento suficiente.
- **Postcondiciones:** El personaje desaparece del mapa visual y pasa a estar dentro del edificio y casilla correspondiente.
- **Escenario Principal:**
 - a. El jugador indica la intención de mover un personaje o grupo hacia una celda que contiene un edificio propio.
 - b. El sistema detecta la interacción con el edificio y evalúa si hay capacidad suficiente para alojar a la(s) unidad(es).
 - c. El sistema introduce al personaje en el edificio, actualizando su estado.
 - d. El sistema retira la representación gráfica del personaje de la celda en el mapa exterior.
- **Escenario Alternativo 1:** Edificio sin capacidad. Si la ciudadela o cuartel ha alcanzado su capacidad máxima de alojamiento, el sistema rechaza la acción, el personaje no puede entrar y permanece en la celda adyacente.
- **Extensiones:** Extiende al caso de uso: “Mover entidad”.
- **Inclusiones:** Incluye al caso de uso: “Refrescar mapa”.
- **Prioridad:** Media

CASO DE USO: CU12 - Atacar

- **Actores:** Jugador
- **Resumen:** Permite a una unidad o grupo de la civilización infligir daño a un personaje, grupo de personajes o edificio de una civilización enemiga que se encuentre dentro de su rango de alcance.
- **Precondiciones:** La partida está iniciada y es el turno del jugador activo. El personaje o grupo atacante cuenta con unidades para atacar y dichas unidades se encuentran en rango de un elemento (personaje o edificio) perteneciente a una civilización rival.
- **Postcondiciones:** La salud del elemento enemigo atacado se reduce según el valor de ataque del agresor, si acaso la salud de dicha entidad se reduce a 0, esta desaparece o si acaso es un edificio pasa a estar destruido.
- **Escenario Principal:**
 - a. El jugador selecciona un personaje o grupo propio.

- b. El jugador indica como objetivo a un elemento enemigo (personaje o edificio) dentro del rango de ataque permitido.
- c. El sistema verifica que el ataque es válido según las reglas de alcance.
- d. El sistema descuenta los puntos de salud correspondientes de la entidad objetivo.
- **Escenario Alternativo 1:** Objetivo fuera de alcance. Si el elemento a atacar no se encuentra en una celda adyacente o en el rango de visión/disparo de la unidad, el sistema cancela la orden e informa del error.
- **Inclusiones:** Incluye al caso de uso: “Refrescar mapa”.
- **Prioridad:** Media

CASO DE USO: CU13 - Eliminar personaje

- **Actores:** Administrador
- **Resumen:** Proceso automático que se encarga de retirar a un personaje o grupo del juego cuando sus puntos de salud se han visto reducidos a cero.
- **Precondiciones:** Un personaje o grupo ha recibido daño producto de un ataque y su nivel de salud ha descendido hasta cero (o menos).
- **Postcondiciones:** El personaje es borrado del registro de unidades de su civilización y desaparece permanentemente del tablero. La celda que ocupaba pasa a estar libre de nuevo.
- **Escenario Principal:**
 - a. Tras un ataque, el sistema verifica el estado de salud del personaje o grupo afectado.
 - b. El sistema identifica que los puntos de vida son iguales o inferiores a cero.
 - c. El sistema procede a eliminar la instancia del personaje o grupo de la partida.
 - d. El personaje o grupo desaparece de la celda que estaba ocupada.
- **Extensiones:** Extiende al caso de uso: “Atacar”.
- **Inclusiones:** Incluye al caso de uso: “Refrescar mapa”.
- **Prioridad:** Media

CASO DE USO: CU14 - Destruir edificio

- **Actores:** Administrador
- **Resumen:** Proceso automático que maneja la demolición de un Edificio cuando su salud llega a cero tras sufrir los ataques de unidades enemigas.
- **Precondiciones:** Un edificio ha sido objetivo de un ataque y sus puntos de vida restantes han llegado a cero.
- **Postcondiciones:** El estado del edificio pasa a ser “destruido” y la celda en la que reside aparece como tal.
- **Escenario Principal:**
 - a. El sistema evalúa la salud del edificio tras procesar un ataque recibido.
 - b. Al comprobar que la salud es cero, el sistema inicia la destrucción del edificio.
 - c. El estado del edificio pasa a ser “destruido” y la celda en la que reside aparece como tal, actualizando el mapa.
- **Escenario Alternativo 1:** Edificio ocupado. Si existían soldados o campesinos protegiéndose en el interior del edificio al momento de su destrucción, el sistema procedía a eliminar también a esos personajes o a expulsarlos a celdas adyacentes.
- **Extensiones:** Extiende al caso de uso: “Atacar”.
- **Inclusiones:** Incluye al caso de uso: “Refrescar mapa”.
- **Prioridad:** Media

CASO DE USO: CU15 - Ser atacado por torre

- **Actores:** Administrador
- **Resumen:** Acción defensiva automática por la cual una torre dispara contra unidades enemigas que realicen acciones dentro de su perímetro de vigilancia.
- **Precondiciones:** Existe una torre construida y operativa. Un personaje o grupo enemigo entra en el radio de acción de dicha torre.
- **Postcondiciones:** El personaje o grupo invasor recibe un daño inmediato calculado a partir de la estadística de ataque base de la torre, pudiendo resultar en su posterior eliminación.
- **Escenario Principal:**
 - a. Una unidad enemiga se sitúa en el rango de ataque de una torre enemiga.
 - b. El sistema interrumpe el flujo normal para procesar la reacción defensiva del edificio.
 - c. La torre ejecuta automáticamente un ataque hacia la entidad invasora.
 - d. El sistema calcula y resta el daño a la salud de la unidad.
- **Escenario Alternativo 1:** Si un grupo entra en la zona, el sistema reparte el daño entre todas las unidades que forman el grupo.
- **Extensiones:** Extiende a los casos de uso: “Mover entidad” y “Recolectar recursos”.
- **Prioridad:** Media

CASO DE USO: CU16 - Reparar edificio

- **Actores:** Jugador
 - **Resumen:** Permite a uno de los paisanos de la civilización activa restaurar los puntos de vida perdidos de un edificio aliado, a cambio de invertir recursos del inventario.
 - **Precondiciones:** La partida está iniciada y es el turno del jugador activo. El jugador selecciona un paisano y un edificio aliado dañado en una celda adyacente. La civilización cuenta con materiales suficientes (madera y/o piedra).
 - **Postcondiciones:** Se restan recursos del saldo de tesorería del jugador y el edificio incrementa sus puntos de salud.
 - **Escenario Principal:**
 - a. El jugador escoge a un paisano y selecciona un edificio propio dañado.
 - b. El jugador ordena la acción de reparación.
 - c. El sistema verifica que la civilización posee los recursos requeridos para el coste de reparación.
 - d. El sistema descuenta la cantidad correspondiente del inventario global.
 - e. Se restituye una cantidad proporcional de puntos de vida al edificio.
 - **Escenario Alternativo 1:** Recursos insuficientes. Si al verificar el inventario en el paso “c” no existen materiales suficientes, el sistema cancela la orden y muestra un mensaje de advertencia.
 - **Inclusiones:** Incluye al caso de uso: “Refrescar mapa”.
 - **Prioridad:** Media
-

2.2 Modelo de Vocabulario

El modelo de vocabulario, tiene como objetivo atrapar los conceptos de mayor relevancia presentes en la realidad a la que se pretende dar soporte, junto con las relaciones que se establecen entre ellos. Se mantiene un nivel de abstracción que no representa software ni aborda el diseño técnico, permitiendo su fácil validación con el cliente.

A continuación se inserta una vista del Modelo de Vocabulario, que plasma la estructura lógica del juego:

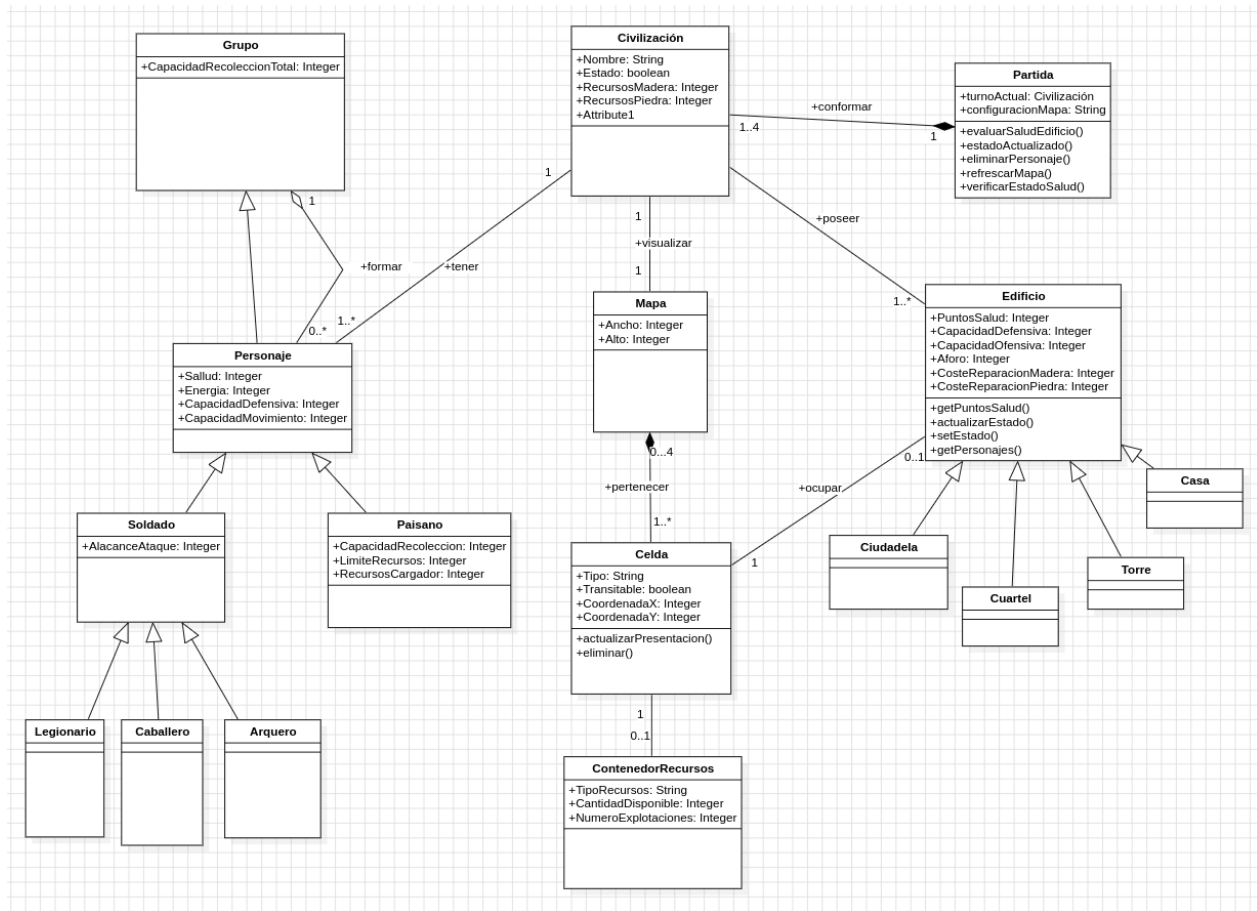


Figure 1: Modelo de vocabulario

Observaciones de interés sobre el modelo:

A partir de la estructura representada en el modelo, destacan las siguientes características del juego:

- **Composición de la Partida:** El concepto central es la **Partida**, la cual se encarga de organizar el entorno. Una partida está conformada por un número de entre uno y cuatro jugadores o facciones (**Civilización**) y visualiza exactamente un único terreno de juego (**Mapa**).
- **Estructura del Entorno:** El **Mapa** se compone a su vez de múltiples unidades geográficas discretas denominadas **Celdas**. Cada celda determina la posición de los elementos y puede llegar a poseer un **ContenedorRecursos** (como bosques, canteras o arbustos).
- **Gestión de Facciones:** Cada **Civilización** gestiona dos grandes tipos de recursos activos en el mapa: **Edificios** y **Personajes**.
- **Entidades e Interacción:** Se establece una fuerte relación mediante la cual un **Edificio** ocupa directamente una **Celda**. Existen múltiples variaciones de estructuras que una facción puede poseer, modeladas como especializaciones lógicas: **Casa**, **Torre**, **Cuartel** y **Ciudadela**.
- **Los Personajes:** Las entidades móviles del juego, englobados bajo el concepto genérico de **Personaje**, se dividen según su especialidad. Por un lado, el **Paisano**, dedicado a tareas

logísticas, y por otro, el Soldado, que a su vez se divide en perfiles de combate específicos (Legionario, Caballero o Arquero).

- **Dinámica de Agrupación:** Un aspecto notable es la capacidad de organizar unidades. Múltiples personajes pueden formar un Grupo, y a efectos lógicos dentro del juego, este grupo es tratado y responde como si fuese un único personaje consolidado.

3. Fase de Construcción

Al iniciar esta fase, el equipo se organizó para seccionar el trabajo y compartimentar las tareas. La complejidad total del proyecto se dividió en tres iteraciones principales, abordando en cada una de ellas una parte o funcionalidad del sistema.

3.1 Planificación

El Modelo de Casos de Uso actuó como la guía principal para estructurar la construcción de la aplicación. El trabajo se organizó según el siguiente calendario de iteraciones y objetivos:

- **Iteración 1 (05/03/26 - 26/03/26): Objetivo:** Dotar al software de la funcionalidad básica.
 - **Casos de Uso:** CU00 - Crear partida, CU01 - Refrescar mapa, CU02 - Mover entidad, CU03 - Construir Edificio, CU04 - Generar Personaje, CU05 - Definir turnos, CU06 - Recolectar recursos, CU06 - Formar grupo, CU07 - Cargar tablero, CU08 - Inscribir jugador, CU09 - Guardar partida, CU10 - Cargar partida.
- **Iteración 2 (26/03/26 - 16/04/26): Objetivo:** Extender y mejorar el rango de acciones posibles, añadiendo la interacción (combate y recolección avanzada) entre los participantes de la partida.
 - **Casos de Uso:** , CU11 - Entrar en edificio, CU12 - Atacar, CU13 - Eliminar personaje, CU14 - Destruir edificio, CU15 - Ser atacado por torre, CU16 - Reparar edificio, CU17 - Almacenar recursos.
- **Iteración 3 (16/04/26 - 07/05/26): * Objetivo:** Completar la funcionalidad faltante, permitiendo el despliegue de información y las mecánicas de grupo.
 - **Casos de Uso:** Imprimir interfaz, Describir información civilización, Mostrar descripción celda, Disolver grupo.

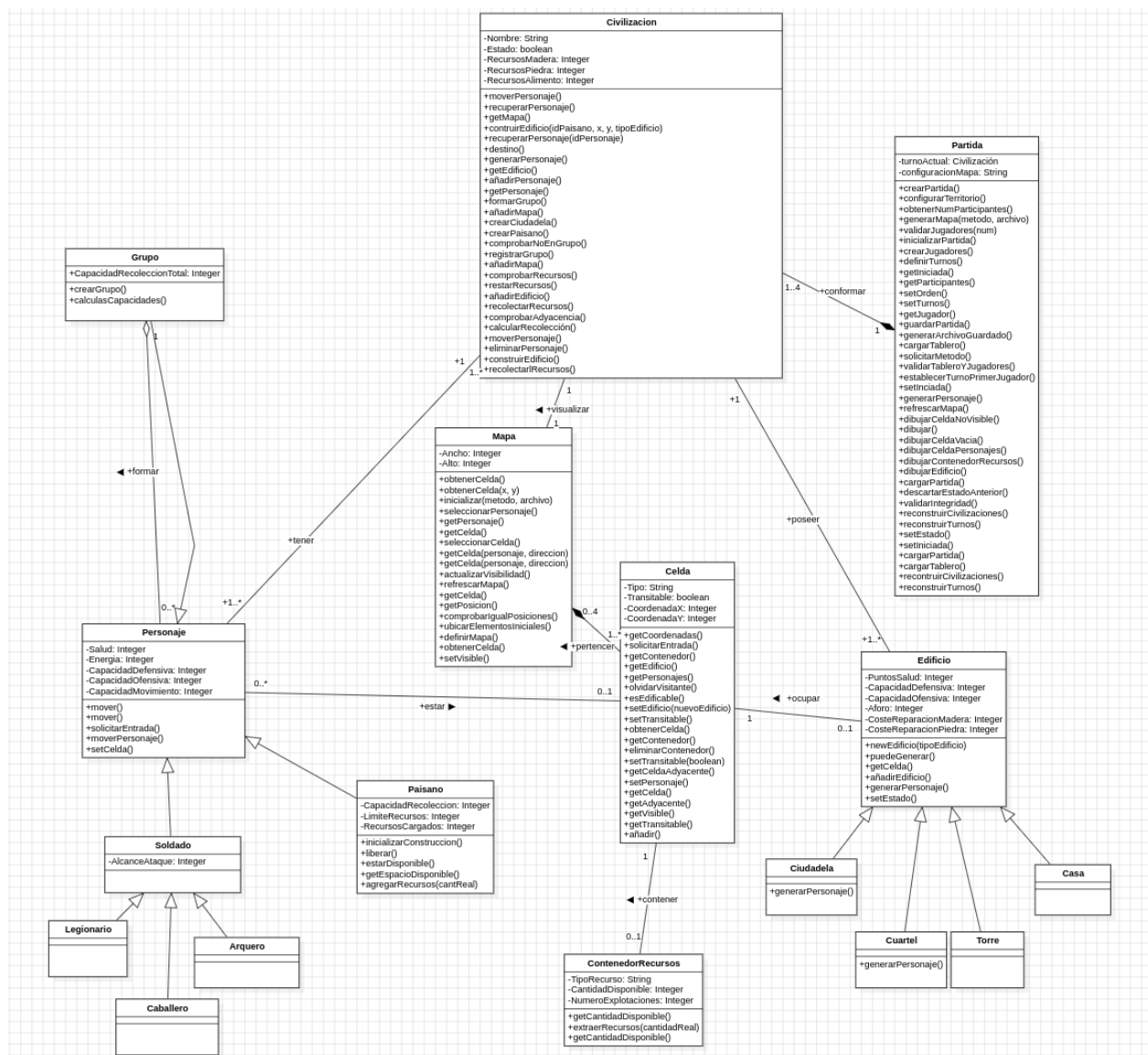
Si bien la planificación del proyecto en un inicio fue esta, por cuestión de calendario no fue posible abordar la implementación de las funcionalidades propias de la iteración número tres.

3.2 Iteración 1

Tal y como se estableció en la planificación, el objetivo de esta primera iteración fue dotar al software de la funcionalidad básica y esencial para jugar una partida. En este ciclo se abordaron exclusivamente los Casos de Uso de Prioridad Alta que definen el núcleo del sistema.

Para llevar a cabo el diseño de esta iteración, el equipo por un reparto equitativo de responsabilidades. Una vez consensuada la estructura de clases inicial, nos dividimos el modelado del comportamiento.

Cada uno de los cuatro participantes del equipo asumió la responsabilidad de diseñar tres diagramas de secuencia correspondientes a los casos de uso de prioridad alta. Esto nos permitió cubrir todo el espectro de funcionalidades básicas de forma eficiente y colaborativa, ilustrando el tiempo de ejecución de las interacciones principales.



3.3 Iteración 2

En la segunda iteración, el objetivo fue extender el rango de acciones posibles, introduciendo nuevas mecánicas como la defensa automática, reparación de edificios o ataque contra entidades y/o estructuras.

El desarrollo de esta iteración supuso un reto importante. Por un lado, fue necesario actualizar los diagramas de secuencia elaborados en la Iteración 1, ya que la inclusión de patrones de diseño alteró el flujo de mensajes y la delegación de responsabilidades entre los objetos. Por otro lado, manteniendo la dinámica de trabajo equitativo, el equipo se repartió el diseño de los nuevos casos de uso de prioridad media, elaborando cada integrante dos nuevos diagramas de secuencia.

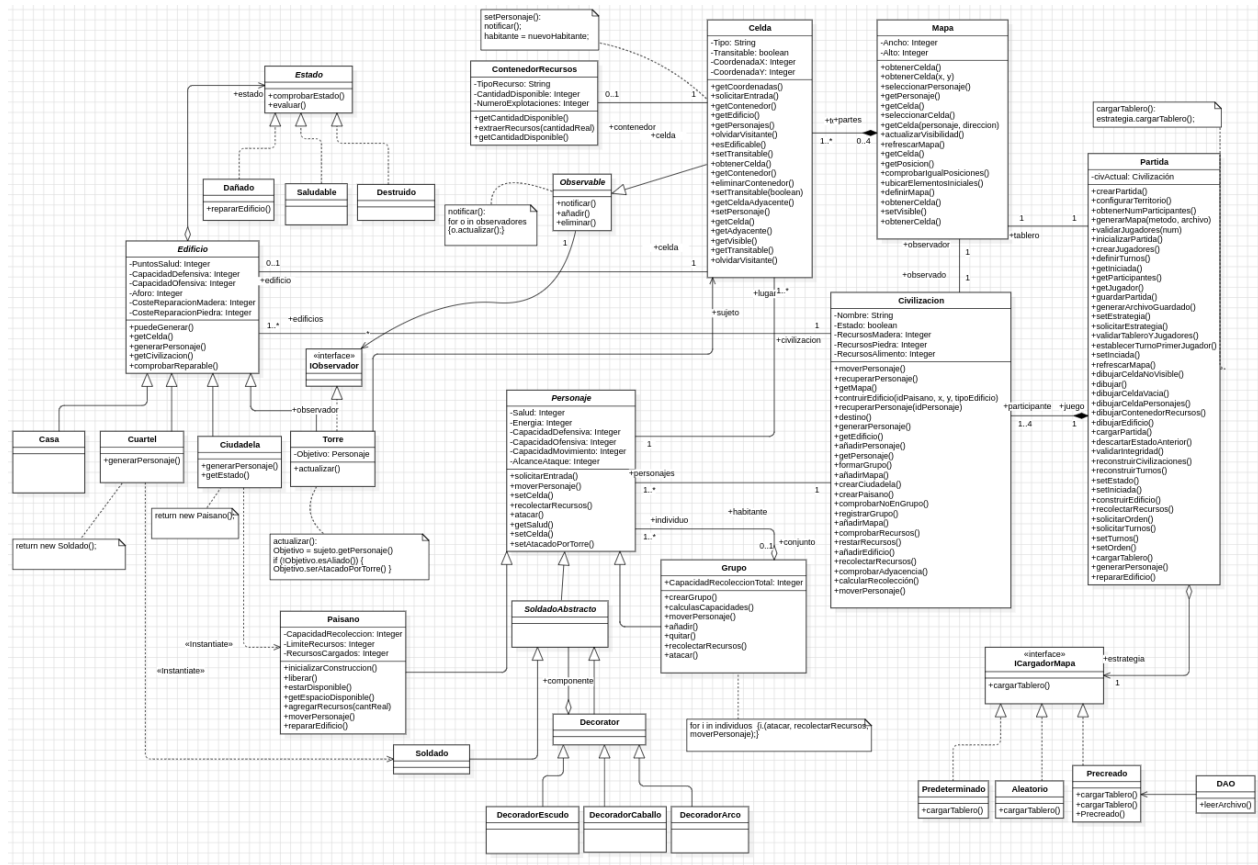


Figure 3: Diagrama segunda iteracion

Con la entrega final a la vuelta de la esquina, el trabajo técnico se centró principalmente en la aplicación de patrones de diseño para resolver problemáticas específicas y mejorar el diseño del sistema mediante la inclusión de patrones de diseño concretos como los siguientes:

1. Patrón Observer (Observador):

- *Objetivo:* Dar soporte al caso de uso “Ser atacado por torre”. La torre actúa como un observador pasivo del terreno. Cuando un personaje cambia de posición, la torre es notificada y, si detecta a un enemigo en su perímetro, dispara automáticamente un ataque sin requerir la intervención del jugador.

2. Patrón State (Estado):

- *Objetivo:* Gestionar el ciclo de vida de las estructuras en los eventos de destrucción

y reparación. Se definió una interfaz **Estado** con transiciones dinámicas a **Saludable**, **Dañado** y **Destruido**. Esto eliminó complejas sentencias condicionales al evaluar si un edificio puede alojar tropas o si su celda debe volver a ser transitable al ser derruido.

3. Patrón Decorator (Decorador):

- *Objetivo:* Proporcionar flexibilidad a la hora de equipar unidades de combate. Mediante un **SoldadoAbstracto** y decoradores concretos (**DecoradorCaballo**, **DecoradorArco**, **DecoradorEscudo**), el sistema modifica dinámicamente los atributos de los soldados en tiempo de ejecución, evitando una explosión combinatoria de subclases.

4. Patrón Composite (Compuesto):

- *Objetivo:* Resolver el caso de uso “Formar grupo”. A través de la clase **Grupo**, el sistema permite al jugador emitir órdenes conjuntas (moverse, atacar, recolectar). El grupo itera sobre sus individuos internos, permitiendo a la Partida tratar a un paisano solitario y a un batallón entero a través de una interfaz de control idéntica.

5. Patrón Strategy (Estrategia):

- *Objetivo:* Desacoplar el algoritmo responsable de “Cargar tablero”. Utilizando la interfaz **ICargadorMapa**, la Partida delega la generación del terreno a estrategias concretas (**Predeterminado**, **Aleatorio**, **Precreado**), facilitando el intercambio de la lógica de creación en tiempo de ejecución.

4. Fase de Transición

En consecuencia, se hace constar que con fecha de 17 de mayo de 2026 se procede a la entrega de la documentación y los archivos del proyecto para que sean sometidos a valoración por parte del profesor. No obstante, el equipo es consciente de que, en un entorno profesional real, esta fase requeriría un tiempo considerable para asegurar la correcta puesta del producto en manos del cliente y garantizar su correcto despliegue.